**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

## Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

### Page 1 onwards · Do Now

Q1. A child key pointing to a nonexistent parent.

Q2. Binary search: O(log n) rather than O(n).

Q3. AND.

Q4. Holding a CPU's working data/ALU results.

### You Do pages · Independent questions

Q1. A Atomicity [1], C Consistency [1], I Isolation [1], D Durability [1], clear atomic/durable explanations [2].

Q2. Both read old value [2], overwriting changes [2], locking/transaction isolation prevents conflict [2].

Q3. Redundancy can reduce downtime [2], RAID is not a backup [2], replication can copy mistakes [2], independent tested backups restore deleted/corrupt data [2].

### Exit Ticket · Model answer

False. Replicated deletions can propagate; an independent backup is still needed.

## Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

**1. Copy the whole original question:**

**2. Write a complete new answer in GREEN PEN. Show working if needed:**





**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

# LESSON 141 · Lesson 141 · ACID Transactions, Locking and Redundancy

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

## Do Now · Recall previous learning

Q1. What does foreign-key referential integrity prevent?

Q2. Which is faster in the worst case on sorted arrays, linear or binary search?

Q3. Which logic gate outputs 1 only when both inputs are 1?

Q4. What is the accumulator for?

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

## I Do · Read new knowledge

### Key vocabulary

transaction — Group of DB operations treated as one unit of work.

ACID — Atomicity, Consistency, Isolation, Durability.

record locking — Restricting conflicting access while data is being changed.

redundancy — Duplicated resources/data for resilience, distinct from backup.

### New learning

Begin transaction — Move £20: subtract account A and add account B.

Protect consistency — Both updates commit together or both roll back.

Coordinate concurrent work — Isolation/locks reduce interference; durable commits survive failures.

### Worked example

A transfer subtracts £20 from Account A but crashes before adding £20 to Account B. Which ACID property matters most?

1. Subtract £20 from A — Identify the relevant information and make a prediction.

2. Crash before adding to B — Explain the change using correct technical terms.

3. Rollback both changes — Atomicity means the transfer is all-or-nothing. A DBMS transaction can roll back partial changes. Consistency checks valid constraints; isolation keeps concurrent transfers safe; durability preserves committed transfers.

## We Do · Practise with guidance

Two users attempt to reserve the last seat at the same time. How do transactions prevent double booking?

Two concurrent requests

Identify the relevant information and make a prediction.

Lock/check last seat

Explain the change using correct technical terms.

Commit one booking

Use an atomic inventory check and reservation update inside an appropriately isolated transaction, with row locking or other concurrency controls. Only one should succeed; second sees no remaining seat.





BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

## You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

### Question 1

Expand ACID and explain atomicity and durability. [6]

### Question 2

Explain why two concurrent users editing one record can cause a lost update. [6]

### Question 3

Evaluate replication, RAID and backups for database resilience. [8]

## Exit Ticket · Answer before checking

RAID or replication guarantees recovery after an accidental DELETE.